

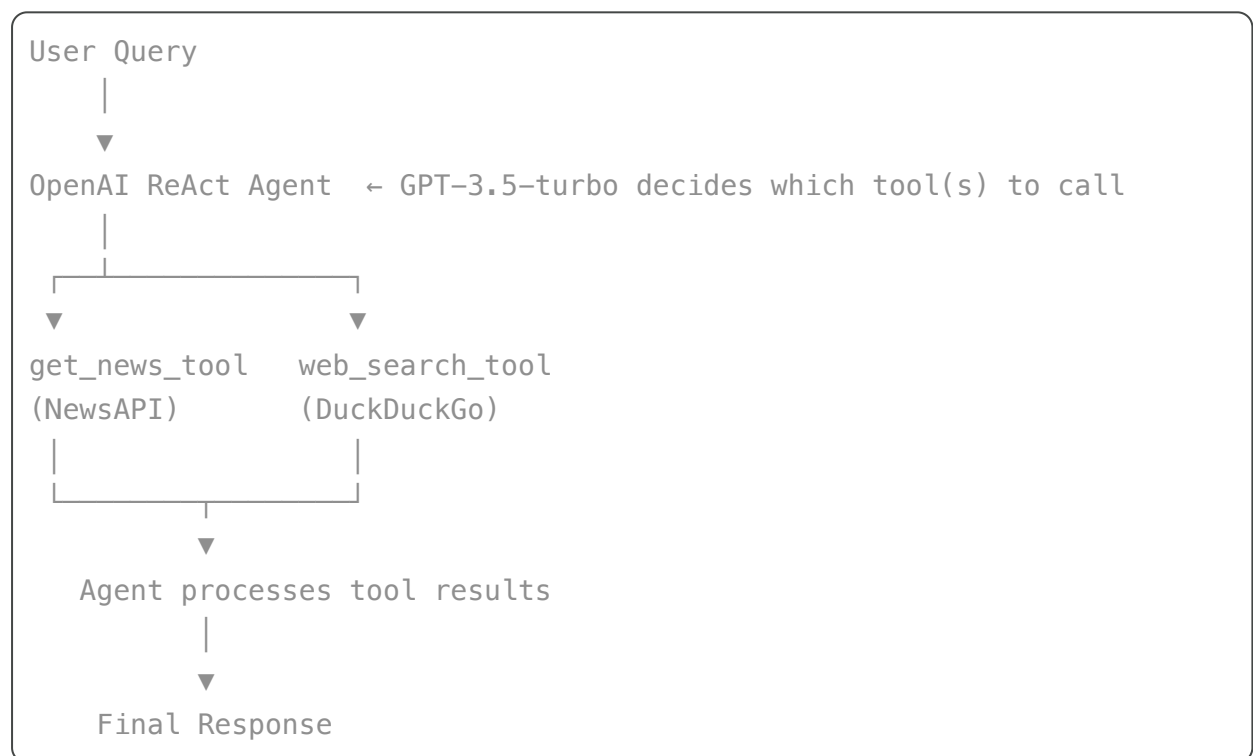


NewsGenie – AI-Powered News & Information Assistant

An end-to-end implementation using:

Component	Technology
Workflow orchestration	LangGraph (<code>create_react_agent</code>)
Language model	OpenAI GPT-3.5-turbo via LangChain (tool-calling)
Real-time news	NewsAPI
Web search (fallback)	DuckDuckGo (no API key needed)
Interactive UI	Streamlit

Architecture



The agent autonomously decides whether to fetch news, search the web, or combine both — no hard-coded routing needed.

```
1 pip install requests==2.32.5
```

```
Requirement already satisfied: requests==2.32.5 in /usr/local/lib/python3.10/site-packages (from -r requirements.txt)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.10/site-packages (from requests==2.32.5)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/site-packages (from requests==2.32.5)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/site-packages (from requests==2.32.5)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/site-packages (from requests==2.32.5)
```

```
1 %pip install -q --upgrade langchain langchain-openai langchain-co
2 streamlit newsapi-python duckduckgo-search python-do
```

✓ Step 1 — Configure API Keys

- **NewsAPI key** – free account at <https://newsapi.org/register>
- **OpenAI key** – <https://platform.openai.com/api-keys>

Enter them in the cell below (input is hidden via `getpass`).

```
1 import os
2 from getpass import getpass
3
4 # — Google Colab: load from Secrets manager —————
5 try:
6     from google.colab import userdata
7     _news_key = userdata.get('NEWS_API_KEY')
8     _openai_key = userdata.get('OPENAI_API_KEY')
9     if _news_key: os.environ['NEWS_API_KEY'] = _news_key
10    if _openai_key: os.environ['OPENAI_API_KEY'] = _openai_key
11 except ImportError:
12     pass # Not running in Colab – fall through to .env / getpass
13
14 # — Local: load from .env file if present —————
15 try:
16     from dotenv import load_dotenv
17     load_dotenv(override=False)
18 except ImportError:
19     pass
20
21 # — Fallback: prompt interactively if still not set —————
22 if not os.getenv('NEWS_API_KEY', '').strip():
23     _k = getpass('Enter your NewsAPI key (or press Enter to skip)')
24     if _k:
25         os.environ['NEWS_API_KEY'] = _k
26
27 if not os.getenv('OPENAI_API_KEY', '').strip():
28     _k = getpass('Enter your OpenAI API key (or press Enter to sk')
29     if _k:
30         os.environ['OPENAI_API_KEY'] = _k
31
32 news_ok = bool(os.getenv('NEWS_API_KEY', '').strip())
33 openai_ok = bool(os.getenv('OPENAI_API_KEY', '').strip())
34 print(f"NewsAPI key : {'✅ set' if news_ok else '❌ NOT SET'}")
35 print(f"OpenAI key : {'✅ set' if openai_ok else '❌ NOT SET'}")
36
```

```
NewsAPI key : ✅ set
OpenAI key : ✅ set
```

✓ Step 2 — Tool Definitions

Two **LangChain tools** (decorated with `@tool`) power the agent:

1. `get_news_tool()` – fetches live headlines from NewsAPI (by category or keyword)
2. `web_search_tool()` – searches the web via DuckDuckGo (zero-config fallback)

The `@tool` decorator automatically generates the JSON schema the OpenAI function-calling API uses to decide which tool to invoke and with what arguments.

```

1 from typing import Optional
2 from newsapi import NewsApiClient
3 from duckduckgo_search import DDGS
4 from langchain_core.tools import tool
5
6 NEWS_CATEGORIES = [
7     'general', 'technology', 'business', 'sports',
8     'health', 'science', 'entertainment',
9 ]
10
11 # — NewsAPI —————
12 @tool
13 def get_news_tool(
14     category: str = 'general',
15     query: Optional[str] = None,
16     page_size: int = 5,
17 ) -> str:
18     """Fetch the latest news headlines from NewsAPI.
19
20     Use this tool when the user asks about current events, breaking
21     top headlines, live scores, stock prices, or anything time-sensitive.
22
23     Args:
24         category: One of general, technology, business, sports, health,
25                 science, or entertainment.
26         query: Optional keyword(s) to filter articles (e.g. 'Apple', 'Tesla').
27         page_size: Number of articles to retrieve (default 5, max 10).
28     """
29     api_key = os.getenv('NEWS_API_KEY', '').strip()
30     if not api_key:
31         return 'NEWS_API_KEY is not configured. Cannot fetch news.'
32     client = NewsApiClient(api_key=api_key)
33     try:
34         if query:
35             resp = client.get_everything(
36                 q=query, language='en', sort_by='relevancy', page_size=page_size
37             )
38         else:
39             resp = client.get_top_headlines(
40                 category=category, language='en', page_size=page_size

```

```

40     articles = [
41         a for a in resp.get('articles', [])
42         if a.get('title') and '[Removed]' not in a.get('title')
43     ]
44     if not articles:
45         return 'No articles found for the given category/query'
46     lines = []
47     for i, a in enumerate(articles, 1):
48         lines.append(
49             f"{i}. **{a.get('title', 'No title')}** - {a.get('description') or 'No description.'}\n"
50             f"URL: {a.get('url', '')}\n"
51             f"Published: {a.get('publishedAt', '')}"
52         )
53     return '\n\n'.join(lines)
54 except Exception as exc:
55     return f'NewsAPI request failed: {exc}'
56
57
58
59 # — DuckDuckGo web search —————
60 @tool
61 def web_search_tool(query: str, max_results: int = 5) -> str:
62     """Search the web via DuckDuckGo and return relevant results.
63
64     Use this tool for factual questions, definitions, historical
65     how-to guides, or any query that does NOT need live news head
66
67     Args:
68         query: The search query to look up.
69         max_results: Maximum number of results to return (default 5)
70     """
71     if not query.strip():
72         return 'No query provided.'
73     try:
74         with DDGS() as ddgs:
75             hits = list(ddgs.text(query, max_results=max_results))
76             if not hits:
77                 return 'No relevant results found.'
78             lines = []
79             for i, r in enumerate(hits, 1):
80                 lines.append(
81                     f"{i}. **{r.get('title', 'No title')}**\n"
82                     f"    {r.get('body', '')}\n"
83                     f"    Source: {r.get('href', '')}"
84                 )
85             return '\n\n'.join(lines)
86     except Exception as exc:
87         return f'Web search failed: {exc}'
88
89 print('Tools defined: get_news_tool, web_search_tool')
90 print(f'  get_news_tool schema: {get_news_tool.name}')
91 print(f'  web_search_tool schema: {web_search_tool.name}')
92

```

```
Tools defined: get_news_tool, web_search_tool
get_news_tool schema: get_news_tool
web_search_tool schema: web_search_tool
```

✓ Step 3 — OpenAI Agent with LangGraph

Instead of a hard-coded pipeline, we use LangGraph's `create_react_agent` to build a **proper OpenAI tool-calling agent**:

What changes	Old approach	New approach
Query routing	Hard-coded <code>classify_query</code> LLM node	Agent autonomously decides which tool(s) to call
Workflow	Fixed 4-node graph	ReAct loop: LLM → tool → LLM → ... → answer
Tool binding	Manual function calls	OpenAI function-calling (JSON schema auto-gene

The `create_react_agent` from `langgraph.prebuilt`:

1. Binds the tools to the LLM via OpenAI function-calling
2. Runs a **ReAct loop** — the model calls tools as needed and reasons over results
3. Stops when the model produces a final answer (no more tool calls)

```
1 from langchain_openai import ChatOpenAI
2 from langchain_core.messages import HumanMessage, SystemMessage
3 from langgraph.prebuilt import create_react_agent
4
5 # — System prompt —————
6 _SYSTEM_PROMPT = (
7     "You are NewsGenie, an AI-powered news and information assist
8     "You have two tools available:\n"
9     " • get_news_tool – fetches live headlines from NewsAPI. U
10    "headlines, current events, live scores, market updates.\n"
11    " • web_search_tool – searches the web via DuckDuckGo. Use f
12    "questions, definitions, historical info, how-to guides.\n\n"
13    "Guidelines:\n"
14    "- Always call a tool before answering – never guess news con
15    "- For news queries, pass the appropriate category and/or key
16    "- Cite sources (article titles, URLs) in your final answer.\n
17    "- Be concise and well-structured."
18 )
19
20 # — LLM —————
21 def _get_llm(temperature: float = 0.3) -> ChatOpenAI:
22     return ChatOpenAI(
23         model='gpt-3.5-turbo',
24         temperature=temperature,
25         api_key=os.getenv('OPENAI_API_KEY', ''),
26     )
27
28 # — Build the ReAct agent graph —————
29 agent_graph = create_react_agent(
30     model=_get_llm(),
31     tools=[get_news_tool, web_search_tool],
```

```

32     prompt=_SYSTEM_PROMPT,
33 )
34
35 print('OpenAI ReAct agent compiled successfully via LangGraph')
36 print(f' Model   : gpt-3.5-turbo')
37 print(f' Tools   : {[t.name for t in [get_news_tool, web_search_t
38

```

```

OpenAI ReAct agent compiled successfully via LangGraph
Model   : gpt-3.5-turbo
Tools   : ['get_news_tool', 'web_search_tool']
/tmp/ipykernel_6114/2762126108.py:29: LangGraphDeprecatedSinceV10: cre
agent_graph = create_react_agent(

```

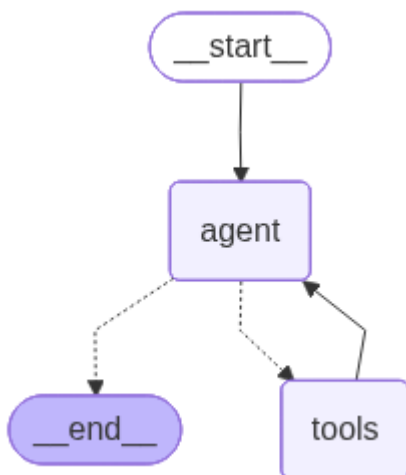
✓ Step 4 — Visualise the Workflow

Renders the state-machine graph as a Mermaid diagram inline.

```

1 from IPython.display import display, Image
2
3 try:
4     display(Image(agent_graph.get_graph().draw_mermaid_png()))
5 except Exception:
6     # Fallback: print Mermaid source
7     print(agent_graph.get_graph().draw_mermaid())
8

```



✓ Step 5 — Test the Agent

Run the ReAct agent against sample queries. The agent autonomously decides which tools to call — no routing logic needed.

```

1 def run_newsgenie(query: str, category: str = 'general') -> dict:
2     """Invoke the NewsGenie ReAct agent.
3

```

```

4     Includes the selected news category as a hint in the human me
5     the agent passes it to get_news_tool when fetching headlines.
6     """"
7     # Embed category context into the query so the agent knows wh
8     augmented_query = f"[Preferred news category: {category}] {qu
9
10    result = agent_graph.invoke({
11        'messages': [HumanMessage(content=augmented_query)]
12    })
13
14    # The last message in the list is always the final AI answer
15    final_response = result['messages'][-1].content
16
17    # Surface which tools were called (for transparency)
18    tools_called = [
19        msg.name
20        for msg in result['messages']
21        if hasattr(msg, 'name') and msg.name
22    ]
23
24    return {
25        'query':      query,
26        'category':   category,
27        'response':   final_response,
28        'tools_called': tools_called,
29        'messages':   result['messages'],
30    }
31
32
33 test_cases = [
34     ('What is machine learning?',      'general'),
35     ("Show me today's top technology headlines", 'technology'),
36     ('Latest sports news',             'sports'),
37 ]
38
39 for query, category in test_cases:
40     print('=' * 70)
41     print(f'Query      : {query}')
42     print(f'Category   : {category}')
43     result = run_newsgenie(query, category)
44     print(f'Tools      : {result["tools_called"]}')
45     print()
46     print(result['response'])
47     print()
48

```

```

=====
Query      : What is machine learning?
Category   : general
/tmp/ipykernel_6114/2755805107.py:74: RuntimeWarning: This package (`
  with DDGS() as ddgs:
sys:1: ResourceWarning: Unclosed socket <zmq.Socket(zmq.PUSH) at 0x7e
ResourceWarning: Enable tracemalloc to get the object allocation trac
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203

```

```

return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
Tools      : ['web_search_tool']

```

Machine learning is a field of artificial intelligence that focuses o

For more information on machine learning, you can visit the following

```

=====
Query      : Show me today's top technology headlines
Category   : technology
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)

```

✓ Step 6 — Interactive Query

Run a single custom query. Edit `my_query` and `my_category` and re-run the cell.

```

1 # Edit query and category, then run this cell
2 my_query      = 'Latest AI research news'
3 my_category    = 'technology'
4

```

```
5 result = run_newsgenie(my_query, my_category)
6 print(f"Tools called : {result['tools_called']}")
7 print()
8 print(result['response'])
9
```

[illegible]

```

return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203

```

✓ Step 7 — Launch Streamlit UI

The next two cells write a self-contained `app.py` (using the same OpenAI ReAct agent pattern) then start the server.

Open <http://localhost:8501> in your browser.

```

1 app_code = '''
2 import os
3 from typing import Optional
4
5 import streamlit as st
6 from dotenv import load_dotenv
7 from newsapi import NewsApiClient
8 from duckduckgo_search import DDGS
9 from langchain_core.tools import tool
10 from langchain_core.messages import HumanMessage
11 from langchain_openai import ChatOpenAI
12 from langgraph.prebuilt import create_react_agent
13
14 load_dotenv()
15
16 NEWS_CATEGORIES = ['general', 'technology', 'business', 'sports',
17                   'health', 'science', 'entertainment']
18
19 _SYSTEM_PROMPT = (
20     "You are NewsGenie, an AI-powered news and information assis
21     "You have two tools:\n"
22     "  • get_news_tool - fetches live headlines from NewsAPI. I
23     "headlines, current events, live scores, market updates.\n"
24     "  • web_search_tool - searches the web via DuckDuckGo. Use
25     "questions, definitions, historical info, how-to guides.\n\
26     "Always call a tool before answering. Cite sources in your f
27 )
28
29
30 @tool
31 def get_news_tool(category: str = "general",
32                  query: Optional[str] = None,
33                  page_size: int = 5) -> str:
34     """Fetch the latest news headlines from NewsAPI.
35
36     Use for news, headlines, current events, live scores, market
37
38     Args:
39         category: One of general, technology, business, sports,
40                 science, or entertainment.
41         query: Optional keyword filter (e.g. "Apple earnings").

```

```

42     page_size: Number of articles (default 5).
43     """
44     api_key = os.getenv("NEWS_API_KEY", "").strip()
45     if not api_key:
46         return "NEWS_API_KEY is not configured."
47     client = NewsApiClient(api_key=api_key)
48     try:
49         if query:
50             resp = client.get_everything(
51                 q=query, language="en", sort_by="relevancy", page
52             )
53         else:
54             resp = client.get_top_headlines(
55                 category=category, language="en", page_size=page
56             )
57         articles = [
58             a for a in resp.get("articles", [])
59             if a.get("title") and "[Removed]" not in a.get("title")
60         ]
61         if not articles:
62             return "No articles found."
63         lines = []
64         for i, a in enumerate(articles, 1):
65             lines.append(
66                 f"{i}. **{a.get('title', '')}** - {a.get('source', '')}\n"
67                 f"    {a.get('description') or ''}\n"
68                 f"    URL: {a.get('url', '')}\n"
69                 f"    Published: {a.get('publishedAt', '')}"
70             )
71         return "\\n\\n".join(lines)
72     except Exception as exc:
73         return f"NewsAPI error: {exc}"
74
75 @tool
76 def web_search_tool(query: str, max_results: int = 5) -> str:
77     """Search the web via DuckDuckGo.
78
79     Use for factual questions, definitions, history, how-to guides, etc.
80
81     Args:
82         query: The search query.
83         max_results: Maximum results to return (default 5).
84     """
85     if not query.strip():
86         return "No query provided."
87     try:
88         with DDGS() as d:
89             hits = list(d.text(query, max_results=max_results))
90             if not hits:
91                 return "No results found."
92             return "\\n\\n".join(
93                 f"{i}. **{r.get('title', '')}**\\n    {r.get('body', '')}\n"
94                 for i, r in enumerate(hits, 1)
95             )
96     except Exception as e:
97         return f"Search failed: {e}"

```

```

97
98
99 @st.cache_resource
100 def get_agent():
101     llm = ChatOpenAI(
102         model="gpt-3.5-turbo",
103         temperature=0.3,
104         api_key=os.getenv("OPENAI_API_KEY", ""),
105     )
106     return create_react_agent(
107         model=llm,
108         tools=[get_news_tool, web_search_tool],
109         prompt=_SYSTEM_PROMPT,
110     )
111
112
113 # — UI —
114 st.set_page_config(page_title="NewsGenie", page_icon="📰", layout="wide",
115                     initial_sidebar_state="expanded")
116
117 st.markdown("""<style>
118 .news-card{background:#f8f9fa;border-left:4px solid #1a73e8;border-right:4px solid #1a73e8;border-top:4px solid #1a73e8;border-bottom:4px solid #1a73e8;padding:12px 16px;margin-bottom:10px;}
119 .news-card h4{margin:0 0 4px 0;color:#1a1a1a;}
120 .news-card p{margin:0;color:#555;font-size:.9rem;}
121 .news-card a{color:#1a73e8;font-size:.85rem;}
122 .badge{display:inline-block;background:#e8f0fe;color:#1a73e8;border:1px solid #cfe2f3;padding:2px 10px;font-size:.8rem;font-weight:600;margin-right:5px;}
123 .user-bubble{background:#1a73e8;color:white;border-radius:16px 16px 0 0;padding:10px 16px;margin:6px 0 6px auto;max-width:80%;}
124 .bot-bubble{background:#f1f3f4;color:#202124;border-radius:16px 16px 0 0;padding:10px 16px;margin:6px auto 6px 0;max-width:80%;}
125 #MainMenu{visibility:hidden;} footer{visibility:hidden;}
126 </style>""", unsafe_allow_html=True)
127
128 if "messages" not in st.session_state:
129     st.session_state.messages = []
130 if "category" not in st.session_state:
131     st.session_state.category = "general"
132
133 with st.sidebar:
134     st.title("📰 NewsGenie")
135     st.caption("AI-powered news & information assistant")
136     st.divider()
137     st.subheader("🗂️ News Category")
138     st.session_state.category = st.selectbox(
139         "Category:", options=NEWS_CATEGORIES,
140         index=NEWS_CATEGORIES.index(st.session_state.category),
141         format_func=str.title,
142     )
143     st.divider()
144     st.subheader("🔑 API Status")
145     nok = bool(os.getenv("NEWS_API_KEY", "").strip())
146     aok = bool(os.getenv("OPENAI_API_KEY", "").strip())
147     st.markdown(f"'✅' if nok else '❌' NewsAPI \n{'✅' if aok else '❌' OpenAI}")

```

```

152     if not (nok and aok):
153         st.warning("Set NEWS_API_KEY and OPENAI_API_KEY in your
154     st.divider()
155     st.subheader("💡 Examples")
156     for ex in ["Latest tech headlines", "Top sports news today",
157               "What is quantum computing?", "Latest business ne
158               "Who invented the internet?"]:
159         if st.button(ex, use_container_width=True, key=f"ex_{ex}"):
160             st.session_state._pending = ex
161     st.divider()
162     if st.button("🗑️ Clear Chat", use_container_width=True):
163         st.session_state.messages = []
164         st.rerun()
165
166 st.markdown("## 📰 NewsGenie – AI-Powered News & Information Ass
167 st.caption("Ask about headlines or any general question. The age
168 st.divider()
169
170 for msg in st.session_state.messages:
171     if msg["role"] == "user":
172         st.markdown(
173             f'\<div class="user-bubble">👤 {msg["content"]}</div>
174             unsafe_allow_html=True)
175     else:
176         st.markdown(
177             \<div class="bot-bubble">🤖 <strong>NewsGenie</stro
178             unsafe_allow_html=True)
179         st.markdown(msg["content"])
180
181 if not st.session_state.messages:
182     st.info("👋 Welcome! Type a query below or pick an example f
183
184 pending = st.session_state.pop("_pending", None)
185 with st.form(key="chat_form", clear_on_submit=True):
186     c1, c2 = st.columns([6, 1])
187     with c1:
188         user_input = st.text_input(
189             "Query", value=pending or "",
190             placeholder="e.g. '\\'Latest AI news\\' or '\\'What is
191             label_visibility="collapsed",
192         )
193     with c2:
194         submitted = st.form_submit_button("Send >", use_containe
195
196 if submitted and user_input.strip():
197     q = user_input.strip()
198     st.session_state.messages.append({"role": "user", "content":
199     with st.spinner("Thinking..."):
200         try:
201             if not os.getenv("OPENAI_API_KEY", "").strip():
202                 raise EnvironmentError("OPENAI_API_KEY is not se
203             augmented = f"[Preferred news category: {st.session_
204             result = get_agent().invoke({
205                 "messages": [HumanMessage(content=augmented)]
206             })

```

```

207         resp_text = result["messages"][-1].content
208     except Exception as exc:
209         resp_text = f"⚠️ Error: {exc}"
210     st.session_state.messages.append({"role": "assistant", "content": resp_text})
211     st.rerun()
212 '''
213
214 with open('app.py', 'w', encoding='utf-8') as fh:
215     fh.write(app_code)
216
217 print('app.py written (OpenAI ReAct agent version).')
218 print('Run: streamlit run app.py')
219

```

```

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)
app.py written (OpenAI ReAct agent version).
Run: streamlit run app.py
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)

```

```

1 import subprocess, sys
2
3 proc = subprocess.Popen(
4     [sys.executable, '-m', 'streamlit', 'run', 'app.py', '--serve
5     stdout=subprocess.PIPE,
6     stderr=subprocess.STDOUT,
7 )
8 print(f'Streamlit server started (PID {proc.pid}).')
9 print('Open http://localhost:8501 in your browser.')
10 print('To stop: proc.terminate()')

```

Streamlit server started (PID 7338).
 Open <http://localhost:8501> in your browser.

```

To stop: proc.terminate()
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)

```

✓ Accessing Streamlit UI with **ngrok**

Since the direct **localhost** link might not be accessible from your browser, we'll use **ngrok** to create a public URL for the Streamlit application.

1. Install **pyngrok**

```
1 pip install pyngrok
```

```

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
    return datetime.utcnow().replace(tzinfo=utc)

```

```

return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)

```

2. Get your **ngrok** Auth Token

- Go to ngrok.com and sign up for a free account.
- Once logged in, go to your dashboard to find your auth token.
- Add this token to Colab's Secrets manager (the '🔑' icon on the left panel) with the name **NGROK_AUTH_TOKEN**.

3. Connect **ngrok** to Streamlit

The Streamlit app is running on port **8501**. We'll use **ngrok** to tunnel this port and get a public URL.

```

1 from google.colab import userdata
2 from pyngrok import ngrok, conf
3
4 # Get ngrok auth token from Colab secrets
5 NGROK_AUTH_TOKEN = userdata.get('NGROK_AUTH_TOKEN')
6
7 if NGROK_AUTH_TOKEN:
8     conf.get_default().auth_token = NGROK_AUTH_TOKEN
9     print("ngrok auth token set.")
10 else:
11     print("NGROK_AUTH_TOKEN not found in Colab secrets. Please add")
12

```

[illegible]

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
```

```
1 ngrok.kill()
```

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203:
    return datetime.utcnow().replace(tzinfo=utc)
```

Now, use the `Streamlit App URL` provided above to access your application in your web browser. If you want to stop the Streamlit server and the `ngrok` tunnel, remember to run `proc.terminate()` and `ngrok.kill()` respectively.

✓ Alternative: Accessing Streamlit UI with `localtunnel`

If you prefer an alternative to `ngrok`, `localtunnel` is another popular choice to create a public URL for your local server. It typically does not require an authentication token.

```
1 # First, install Node.js and npm (usually already available in Co
2 # Then, install localtunnel globally using npm
3 !npm install -g localtunnel
```

Now, let's start `localtunnel` on port `8501` where the Streamlit app is running.

```
1 import subprocess
2 import threading
3 import time
4
5 def run_localtunnel():
6     # Use the localtunnel command directly
7     command = ['lt', '--port', '8501']
8     process = subprocess.Popen(command, stdout=subprocess.PIPE, s
9
10     # Wait a bit for localtunnel to start and print its URL
11     time.sleep(5)
12
13     stdout, stderr = process.communicate(timeout=15)
```

```

14     print(stdout)
15     if stderr:
16         print(f"Error from localtunnel: {stderr}")
17
18 # Run localtunnel in a separate thread to avoid blocking the note
19 localtunnel_thread = threading.Thread(target=run_localtunnel)
20 localtunnel_thread.start()
21 print("Localtunnel is starting. Please wait for the public URL to
22 print("You might need to manually check the output or re-run this
23

```

```

1 # Install nbconvert with the webpdf extra (includes Chromium supp
2 %pip install -q "nbconvert[webpdf]"
3

```

```

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)

```

```
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203
```

You should see the public `localhosttunnel` URL in the output above. Copy that URL and paste it into your browser to access the Streamlit application. To stop the `localhosttunnel` process, you may need to interrupt the kernel or restart the runtime.